

IPC144 – Introduction to C

Programming

Practice Exercises

Week 4

Q1 – (Parking Charges) A parking garage charges a \$2.00 minimum fee to park for up to three hours and an additional \$0.50 per hour for each hour or part thereof over three hours. The maximum charge for any given 24-hour period is \$10.00. Assume that no car parks for longer than 24 hours at a time. Write a program that will calculate and print the parking charges for each of three customers who parked their cars in this garage yesterday. You should enter the hours parked for each customer. Your program should print the results in a neat tabular format and should calculate and print the total of yesterday's receipts. The program should use the function `calculateCharges` to determine the charge for each customer. Your outputs should appear in the following format:

Car	Hours	Charge
1	1.5	2.00
2	4.0	2.50
3	24.0	10.00
TOTAL	29.5	14.50

Q2 – (Rounding Numbers) An application of function `floor` is rounding a value to the nearest integer. The statement

```
y = floor( x + .5 );
```

will round the number `x` to the nearest integer and assign the result to `y`. Write a program that reads several numbers and uses the preceding statement to round each of these numbers to the nearest integer. For each number processed, print both the original number and the rounded number.

Q3 – (Rounding Numbers) Function `floor` may be used to round a number to a specific decimal place. The statement

```
y = floor( x * 10 + .5 ) / 10;
```

rounds `x` to the tenths position (the first position to the right of the decimal point). The statement

```
y = floor( x * 100 + .5 ) / 100;
```

rounds x to the hundredths position (the second position to the right of the decimal point). Write a program that defines four functions to round a number x in various ways

- a) roundToInteger(number)
- b) roundToTenths(number)
- c) roundToHundredths(number)
- d) roundToThousandths(number)

For each value read, your program should print the original value, the number rounded to the nearest integer, the number rounded to the nearest tenth, the number rounded to the nearest hundredth, and the number rounded to the nearest thousandth.

Q4 – Answer each of the following questions.

- a) What does it mean to choose numbers “at random”?
- b) Why is the rand function useful for simulating games of chance?
- c) Why would you randomize a program by using srand? Under what circumstances is it desirable not to randomize?
- d) Why is it often necessary to scale and/or shift the values produced by rand?
- e) Why is computerized simulation of real-world situations a useful technique?

Q5 – Write statements that assign random integers to the variable n in the following ranges:

- a) $1 \leq n \leq 2$
- b) $1 \leq n \leq 100$
- c) $0 \leq n \leq 9$
- d) $1000 \leq n \leq 1112$
- e) $-1 \leq n \leq 1$
- f) $-3 \leq n \leq 11$

Q6 – For each of the following sets of integers, write a single statement that will print a number at random from the set.

- a) 2, 4, 6, 8, 10.
- b) 3, 5, 7, 9, 11.
- c) 6, 10, 14, 18, 22.

Q7 – (Hypotenuse Calculations) Define a function called hypotenuse that calculates the length of the hypotenuse of a right triangle when the other two sides are given. Use this function in a program to determine the length of the hypotenuse for each of the following triangles. The function should take two arguments of type double and return the hypotenuse as a double.

Q8 – (Exponentiation) Write a function integerPower(base, exponent) that returns the value of $\text{base}^{\text{exponent}}$

For example, `integerPower( 3, 4 ) = 3 * 3 * 3 * 3`. Assume that exponent is a positive, nonzero integer, and base is an integer. Function integerPower should use for to control the calculation. Do not use any math library functions.

Q9 – (Multiples) Write a function multiple that determines for a pair of integers whether the

second integer is a multiple of the first. The function should take two integer arguments and return 1 (true) if the second is a multiple of the first, and 0 (false) otherwise. Use this function in a program that inputs a series of pairs of integers.

Q10 – (Even or Odd) Write a program that inputs a series of integers and passes them one at a time to function even, which uses the remainder operator to determine if an integer is even. The function should take an integer argument and return 1 if the integer is even and 0 otherwise.

Q11 – (Parking Charges) Write a function that displays a solid square of asterisks whose side is specified in integer parameter side. For example, if side is 4, the function displays:

```
****
****
****
****
```

Q12 – (Displaying a Square of Any Character) Modify the function created in Previous Exercise to form the square out of whatever character is contained in character parameter fillCharacter. Thus if side is 5 and fillCharacter is “#” then this function should print:

```
#####
#####
#####
#####
#####
```

Q13 – (Project: Drawing Shapes with Characters) Use techniques similar to those developed in Previous Exercises to produce a program that graphs a wide range of shapes.

Q14 – (Separating Digits) Write program segments that accomplish each of the following:

- Calculate the integer part of the quotient when integer a is divided by integer b.
- Calculate the integer remainder when integer a is divided by integer b.
- Use the program pieces developed in a) and b) to write a function that inputs an integer between 1 and 32767 and prints it as a series of digits, with two spaces between each digit.

For example, the integer 4562 should be printed as:

4 5 6 2

Q15 – (Time in Seconds) Write a function that takes the time as three integer arguments (for hours, minutes, and seconds) and returns the number of seconds since the last time the clock “struck 12.” Use this function to calculate the amount of time in seconds between two times, both of which are within one 12-hour cycle of the clock.

Q16 – (Temperature Conversions) Implement the following integer functions:

- a) Function `celsius` returns the Celsius equivalent of a Fahrenheit temperature.
- b) Function `fahrenheit` returns the Fahrenheit equivalent of a Celsius temperature.
- c) Use these functions to write a program that prints charts showing the Fahrenheit equivalents of all Celsius temperatures from 0 to 100 degrees, and the Celsius equivalents of all Fahrenheit temperatures from 32 to 212 degrees. Print the outputs in a neat tabular format that minimizes the number of lines of output while remaining readable.

Q17 – (Find the Minimum) Write a function that returns the smallest of three floating-point numbers.

Q18 – (Perfect Numbers) An integer number is said to be a perfect number if its factors, including 1 (but not the number itself), sum to the number. For example, 6 is a perfect number because $6 = 1 + 2 + 3$. Write a function `perfect` that determines if parameter number is a perfect number. Use this function in a program that determines and prints all the perfect numbers between 1 and 1000.

Print the factors of each perfect number to confirm that the number is indeed perfect. Challenge the power of your computer by testing numbers much larger than 1000.

Q19 – (Prime Numbers) An integer is said to be prime if it's divisible by only 1 and itself. For example, 2, 3, 5 and 7 are prime, but 4, 6, 8 and 9 are not.

- a) Write a function that determines if a number is prime.
- b) Use this function in a program that determines and prints all the prime numbers between 1 and 10,000. How many of these 10,000 numbers do you really have to test before being sure that you have found all the primes?
- c) Initially you might think that $n/2$ is the upper limit for which you must test to see if a number is prime, but you need go only as high as the square root of n . Why? Rewrite the program, and run it both ways. Estimate the performance improvement.